

January 2026 - 3

A Real MongoDB App, Ported to JSONB



Figure 1: Figure 0: A Real MongoDB App, Ported to JSONB

Welcome to this edition of yugabyteDB Developer’s Notebook (YDN). This month we answer the following question(s);

We have a real MongoDB application – MongoDB’s own MFlix sample movie database, a Java Spring Boot service plus a web frontend – and we’re evaluating yugabyteDB. Do we have to go through the MongoDB-compatible API to make this work, or is there a more direct path ? And what does real MongoDB-shaped data (deeply nested, schema-varying documents) actually look like once it lands in yugabyteDB ?

There are genuinely two paths, and this month’s article covers both, using a real, running application rather than a synthetic example. The MongoDB-compatible API path hits the same secondary-index gap covered in December’s article. The path this application actually uses in production is more direct: store each MongoDB document as a single JSONB column in an ordinary YSQL table, behind a service layer that can target either backend. We cover the real schema design (including a generated-column trick that’s easy to get wrong on first read), four different ways to index a JSONB column, and a real query-performance incident this project hit and fixed.

Software versions The primary software components used in this edition of YDN include yugabyteDB Anywhere (YBA) and yugabyteDB (YB) version 2026.1.1.0, Java / Spring Boot, and Next.js/TypeScript for the frontend. All of the steps below are run on one very large sized Mac Book Pro, or if you prefer, run these steps on yugabyteDB Aeon, yugabyteDB’s managed service, or Amazon Web Services (AWS), Google Cloud Platform (GCP), or another hyper-scaler.

For isolation and (simplicity), we develop and test all systems inside virtual machines or emulated machines, using the desktop hypervisors VMware Fusion version 13.6.4, and/or UTM version 4.6.5. Generally we run a single node of YBA, and 8 nodes of YB (5 nodes in one region, and 3 nodes in a second region) using Alma Linux versions 8 and 9, and Ubuntu Desktop version 24.04. We also run a client node using Ubuntu Desktop version 24.04.

3.1 Terms and core concepts

3.1.1 Introduction: Two Real Paths for the Same Application

This month's project is MongoDB's own MFlix sample application (a movie catalog, 23,530 documents in `movies`, 1,473 in a vector-embedding-enriched `embedded_movies`, 50,304 in `comments`) – a real Java Spring Boot service and web frontend, modified to run against either MongoDB itself or yugabyteDB, chosen at runtime by the user.

Pointed at yugabyteDB through the MongoDB-compatible API (FerretDB + DocumentDB, the same stack covered in December's article), the application's own startup verification step surfaces the exact gap that article covered in detail – confirmed directly in this project's own logs:

Example 3-1: Real startup output against yugabyteDB's MongoDB API

```
Could not create text search index: Command failed with error 1 (InternalError):
'access method "documentdb_rum" does not exist'
...
```

Aggregation queries filtering by year may be slower without the index

This isn't a hypothetical – it's the same missing-RUM-access-method gap from December's article, now observed from the outside, as an ordinary application startup warning rather than from inside DocumentDB's own source. The application degrades gracefully (it logs a warning and continues), but confirms plainly that index-dependent features (text search, year filtering, comment lookups) are affected without the Option C-style secondary index work covered last month.

The second path – the one actually used for this project's yugabyteDB comparisons – skips the MongoDB wire protocol entirely: store each document as a single JSONB column in an ordinary YSQL table, and write YSQL queries directly against it. No RUM dependency, because there's no MongoDB-index emulation layer involved at all.

3.1.2 The Schema: One JSONB Column, Several Generated Companions

Example 3-2: The real movies table

```
CREATE TABLE movies
(
  id          uuid NOT NULL DEFAULT gen_random_uuid(),
  doc         jsonb NOT NULL,
  mongo_id    text GENERATED ALWAYS AS (doc ->> '{_id,$oid}') STORED,
  title       text GENERATED ALWAYS AS (doc ->> 'title') STORED,
  year_text   text GENERATED ALWAYS AS (COALESCE(doc #>> '{year,$numberInt}', doc ->> 'year')) STORED,
  CONSTRAINT movies_mongo_id_key UNIQUE (mongo_id),
  PRIMARY KEY (id HASH)
);
```

The full MongoDB document lands, unmodified, in `doc`. Three additional columns are `GENERATED ALWAYS AS (...)` `STORED` – computed automatically from an expression over `doc` on every insert or update, impossible to write to directly, and physically stored on disk exactly like an ordinary column. This is a deliberate, explicit tradeoff, worth being precise about: `title` genuinely duplicates data already present inside `doc`. The reason to accept that redundancy is query performance – `WHERE title = 'Inception'` against a real column can use an ordinary index and never has to re-parse JSON; `WHERE doc->>'title' = 'Inception'` without a supporting structure would need to extract and compare that JSON path on every row scanned.

`year_text`'s expression is worth reading closely, because it reflects a genuine data-shape inconsistency in the source MongoDB export itself: `COALESCE(doc #>> '{year,$numberInt}', doc ->> 'year')`. Some documents store `year` as a MongoDB extended-JSON typed wrapper (`{"$numberInt": "1994"}`), reached via the two-level path `#>>`, while others store it as a plain scalar (reached directly via `->> 'year'`). The

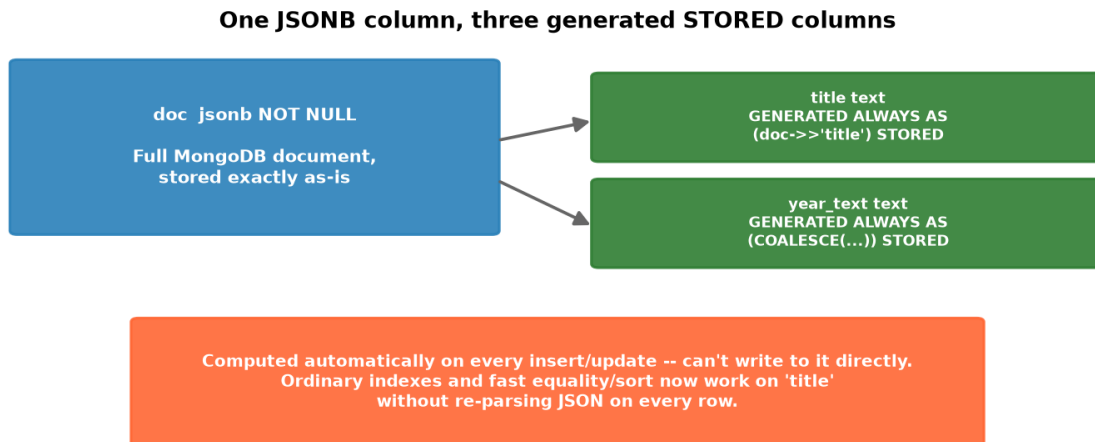


Figure 2: Figure 1: One JSONB column, three generated STORED columns

generated column absorbs that inconsistency once, in the schema, rather than requiring every query written against this table to handle both shapes itself.

3.1.3 Exploring a Schema You Don't Fully Know Ahead of Time

A genuine advantage of JSONB over a rigid, pre-declared schema is direct introspection – useful precisely because MongoDB collections don't enforce a single fixed shape across every document.

Example 3-3: What keys actually exist, and what type is each one

```

-- All distinct top-level keys actually present across every row:
SELECT DISTINCT jsonb_object_keys(doc) AS key FROM movies;
-- 22 rows: _id, fullplot, cast, title, awards, released, metacritic,
-- languages, rated, year, writers, runtime, poster, plot, countries,
-- tomatoes, directors, imdb, lastupdated, genres, type, num_mflix_comments

-- What JSON type does each key actually hold?
SELECT key, jsonb_typeof(doc -> key) AS type
FROM movies, jsonb_object_keys(doc) AS key
GROUP BY key, type
ORDER BY key;
-- cast          / array
-- fullplot      / string
-- imdb          / object
-- runtime       / object    -- itself a wrapped {"$numberInt": "..."} value
-- title         / string
-- ...

```

This project also maintains a second, related collection, `embedded_movies` (1,473 documents, each carrying a `plot_embedding` vector alongside the same movie fields), used for a separate vector-search comparison. A

real, verified cross-check confirmed referential consistency between the two before relying on either: every title in `embedded_movies` also exists somewhere in `movies` –

Example 3-4: Confirming zero orphaned titles

```
SELECT COUNT(*)
FROM embedded_movies e
WHERE (e.doc->>'title') NOT IN (
    SELECT m.doc ->> 'title' FROM movies m WHERE m.doc->>'title' IS NOT NULL
);
-- count
-- -----
--      0
```

3.1.4 Four Ways to Index a JSONB Column

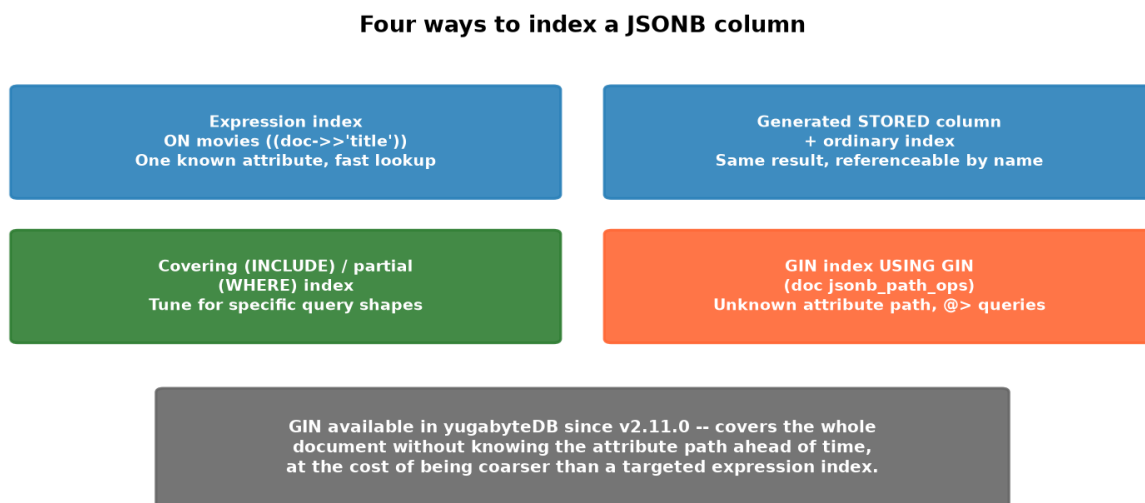


Figure 3: Figure 2: Four ways to index a JSONB column

Expression index – one known attribute path. Rather than adding a generated column, an index can be built directly on an expression:

```
CREATE INDEX ON movies ((doc->>'title'));
```

This makes `WHERE doc->>'title' = 'Inception'` use an Index Scan instead of scanning and re-parsing every row's JSON. Unique constraints work the same way (`CREATE UNIQUE INDEX ON movies ((doc->>'title'))`), and both covering indexes (`INCLUDE`) and partial indexes (`WHERE`) compose normally on top of an expression index – the same tuning tools available for an ordinary column, just applied to a JSON path instead.

Generated STORED column plus an ordinary index – the same result, referenceable by name.

This is section 3.1.2's approach: functionally equivalent query performance to an expression index, but the computed value gets a real column name other queries and other developers can reference directly, at the cost of the redundant on-disk storage discussed above.

GIN index with jsonb_path_ops – an attribute path you don’t know ahead of time. For querying across arbitrary, unpredictable keys inside a document rather than one specific known path:

```
CREATE INDEX ON movies USING GIN (doc jsonb_path_ops);
-- Enables the containment operator:
SELECT * FROM movies WHERE doc @> '{"title": "Inception"}';
```

A GIN index (Generalized Inverted Index) covers the whole document (or a subtree) without needing to know the exact attribute path at index-creation time, and supports the @> (containment), @?, and @@ operators – at the cost of being coarser and generally larger than a targeted expression index built for one specific, known attribute. GIN indexes have been available in yugabyteDB since v2.11.0.

[!NOTE] None of these four approaches is universally “correct” – they trade off differently. A known, frequently-filtered attribute (**title**, **year**) is a good fit for an expression index or a generated column; an attribute whose presence or path varies unpredictably across documents, or a query that needs to search broadly across a document’s structure, is a better fit for GIN. This project uses generated columns for its two or three hottest, most predictable fields, and leaves everything else to be queried through **doc** directly, un-indexed, since those paths aren’t hit often enough to justify the maintenance cost.

3.1.5 A Real Query-Performance Incident

Not every part of this port was smooth on the first attempt. The application’s aggregations page – a report of movies together with their comment activity – originally built its query starting from **movies**, performing a \$lookup-equivalent join into **comments**, then sorting the joined result. Against the real dataset (23,530 movies, 50,304 comments), this pattern was expensive enough that the frontend’s own 15-second aggregation timeout was hit on it, aborting the request outright.

The fix reversed the pipeline’s starting point: begin from **comments** (sorted newest-first), group by movie, apply the limit *before* ever touching **movies**, and only then join the small resulting set of movie IDs back to their full movie records. The report needed to answer “which movies have the most recent comment activity” – structuring the query to start from the smaller, already-sorted side of the join and only pull in the larger table’s full rows for the few movies that survive, rather than joining everything first and sorting after, produced the same correct report without needing the timeout raised at all (though the frontend timeout was also raised afterward, as a second, independent safety margin, not a substitute for the query fix).

3.2 Complete the following

In this section, we build the core JSONB schema, populate it, and reproduce two of the four indexing strategies from section 3.1.4 directly.

3.2.1 Prerequisites

A yugabyteDB cluster and **psql** access. A sample of MongoDB extended-JSON documents (MongoDB’s own publicly available **sample_mflix.movies** export works directly, or any similarly-shaped JSON documents).

3.2.2 Step 1 – Create the schema and load documents

Example 3-5: Standing up the table from Example 3-2, then loading data

```
CREATE TABLE movies (
  id          uuid NOT NULL DEFAULT gen_random_uuid(),
  doc         jsonb NOT NULL,
  title       text GENERATED ALWAYS AS (doc ->> 'title') STORED,
  PRIMARY KEY (id HASH)
);
```

```
-- Loading is a single INSERT per JSON document, e.g.:
INSERT INTO movies (doc) VALUES ('{"title": "Inception", "year": 2010, ...}':::jsonb);
```

3.2.3 Step 2 – Confirm the generated column behaves as documented

Example 3-6: Attempting to write to a generated column directly

```
UPDATE movies SET title = 'Something Else' WHERE title = 'Inception';
-- ERROR: column "title" can only be updated to DEFAULT
```

Confirm this fails exactly as section 3.1.2 describes, then confirm the *correct* way to change the underlying data – update doc itself, and watch title recompute automatically:

```
UPDATE movies SET doc = jsonb_set(doc, '{title}', '"Inception Redux"')
WHERE title = 'Inception';
```

```
SELECT title FROM movies WHERE doc->>'title' = 'Inception Redux';
```

3.2.4 Step 3 – Build and compare an expression index and a GIN index

Example 3-7: Both indexing strategies from section 3.1.4

```
-- Expression index on a known, frequently-queried attribute:
CREATE INDEX movies_title_expr_idx ON movies ((doc->>'title'));

-- GIN index for containment queries across unknown/varying paths:
CREATE INDEX movies_doc_gin_idx ON movies USING GIN (doc jsonb_path_ops);
```

Run EXPLAIN against a title-equality query and confirm the expression index produces an Index Scan; run EXPLAIN against a containment query (WHERE doc @> '{"rated": "PG"}') and confirm the GIN index is used instead. Confirm neither index makes the other unnecessary – they serve different query shapes, exactly as described in section 3.1.4.

Section summary In this section, we built the real generated-column schema this project uses, confirmed the specific, sometimes-surprising behavior of a **GENERATED ALWAYS AS ... STORED** column (computed automatically, not directly writable), and built both an expression index and a GIN index side by side, confirming each serves a genuinely different query shape rather than one simply superseding the other.

3.3 In this document, we reviewed or created

This month and in this document we detailed the following:

- A real MongoDB application (MFlix) confirmed, from the outside, to hit the same MongoDB-compatible-API secondary-index gap covered in depth in December’s article – and the alternative, more direct path this project actually uses in production: JSONB in ordinary YSQL tables, behind a service layer that can target either MongoDB or yugabyteDB.
- The real schema behind that path: a single jsonb document column plus **GENERATED ALWAYS AS ... STORED** companion columns for the handful of fields that are queried, sorted, or filtered on often enough to justify the redundancy – including a real data-shape inconsistency (**year** stored two different ways across documents) absorbed once in the schema via **COALESCE**.
- Direct schema introspection over a collection of documents that don’t share one single fixed shape (jsonb_object_keys, jsonb_typeof), plus a real, verified referential-integrity cross-check between two related collections.

- Four distinct JSONB indexing strategies – expression index, generated column plus ordinary index, covering/partial variants of either, and GIN with `jsonb_path_ops` – each suited to a different combination of “how well-known is the attribute path” and “how is it actually queried.”
- A real query-performance incident (an aggregation report hitting a 15-second frontend timeout) and the actual fix: restructuring the join to start from the smaller, already-sorted side rather than simply raising the timeout.

Persons who helped this month Kiyu Gabriel, David Bechberger

Additional resources: Free yugabyteDB training courses,

https://university.yugabyte.com/users/sign_in

Take any class, anytime, for free.

Jim Knicely’s very excellent blog site,

<https://yugabytedb.tips/>

This document is located here,

<https://www.yugabitten.com/62%20-%20Monthly%20articles/2026-01%20-%20JSONB%20Document%20Modeling/01%20JSONB%20document%20modeling.pdf>